Projeto CubeSat NASCERR

Payload: NASCERR (Nanosatellite Error Resilient Radiation Mission)

DOCUMENTO DE CONTROLE DE INTERFACE

Aprovadores:

_____

Jarbas Aryel Nunes da Silveira
Project Manager, LESC

_____

Wendell Pereira da Silva
Embedded Systems Program Manager, Atlas Software

1. INTRODUÇÃO 4
1.1. Visão Geral 4
1.2. Interfaces 5
2. INTERFACE FÍSICA 6
2.1. Conector 6
2.2. Sinais 6
2.3. Energização e Inicialização 7
2.4. Desenergização 8
3. PROTOCOLO DE COMUNICAÇÃO NASCERR 9
3.1. Introdução 9
3.2. Especificações Gerais 9
3.3. Mapa de Registradores 10
STAT – Registro de status de operação 11
IDX – Registro do Número de Bytes a Serem Transmitidos 14
TXB – Registro de Dados de Telemetria 14
CMD – Registro de Telecomando do Experimento 15
SLEEP – Registro para Entrada no Modo de Economia de Energia 16
WAKEUP – Registro para Saída do Modo de Economia de Energia 16
GPS – Registro de Dados GNSS e Localização 17
HW_STAT – Registro de Status de Hardware 18
HW_TXB – Registro de Dados de Hardware 18
BOOT_DATA – Registro Transmissão de Dados para Atualização de Firmware 19
BOOT_STAT – Registro de Status de Bootloader 21
EXP_STAT– Registro de Sumário de Experimentos Executados 22
EXP_SUM_IDX – Registro de Índice de Páginas de Sumário 23
EXP_SUM_DATA – Registro de Dados de Sumário de Experimentos 23
DB_CLEAR – Registro de Apagamento de Estruturas de Dados 24
LOG_SUM – Registro de sumário de logs 25
LOG_SET_PAGE – Registro para selecionar a página de logs a ler 25
LOG_READ_PAGE – Registro para ler o conteúdo da página com logs selecionada 26
3.4. Telecomandos do NASCERR 28
Formato do Telecomando 29
Escrita/Agendamento (WRITE_CMD = 0x01) 30

Leitura (READ_CMD = 0x00) 30
3.5. Telemetria NASCERR 31
Formato da Telemetria 31
Fluxo de Telemetria (leitura de resultado) 32
Detalhamento por fase 32
3.6. Fluxo de Realização do Experimento 33
3.7. Processo de Atualização de Firmware 34
Etapas do processo 34
Cálculo do Hash SHA-256 36
3.8. Interação do Sistema 37
MASTER 37
SLAVE 38
4. ACRÔNIMOS 39
5. REFERÊNCIAS 41

1. INTRODUÇÃO

1.1. Visão Geral

Este documento descreve as interfaces físicas e lógicas do experimento NASCERR (*Nanosatellite Error Resilient Radiation Mission*), desenvolvido pelo Laboratório de Engenharia de Sistemas de Computação (LESC) da Universidade Federal do Ceará (UFC). A missão tem como objetivo validar o sistema ROBC (*Robust On-Board Computer*), um sistema de computação embarcada robusto para CubeSats, projetado para operar em ambientes de alta radiação em Órbita Baixa da Terra (LEO).

O ROBC utiliza um FPGA SmartFusion2 M2S025, configurado como um bloco intermediário (*Man-in-the-Middle*) entre o processador e a memória SRAM. Esta arquitetura inovadora permite a correção de erros de forma transparente, aplicando algoritmos dinâmicos de ECC (Código de Correção de Erros). O principal desafio técnico abordado pelo NASCERR é a mitigação dos efeitos adversos causados por partículas de alta energia, como prótons e partículas alfa, que interagem com o substrato de silício dos transistores, gerando eventos de ionização. Esse fenômeno pode induzir efeitos transitórios em componentes sensíveis, como as células de memória SRAM, comprometendo a integridade dos dados.

Resumidamente, o objetivo científico da missão é validar um sistema de proteção de dados que garanta a confiabilidade de satélites em ambientes extremos, contribuindo para avanços significativos em missões espaciais de baixo custo e alta resiliência. Os testes realizados em condições simuladas de radiação e variação térmica demonstraram recuperação de eventos de bitflip, além de manter desempenho consistente em tarefas críticas, como processamento de imagens e convoluções.

1.2. Interfaces

O NASCERR pode ser acoplada a um CubeSat 2U, utilizando uma interface CSKB padrão que fornece alimentação elétrica, sinais de controle e telemetria.

A interface lógica utiliza um protocolo de comunicação sobre I2C, organizando as informações na forma de banco de registradores lógicos.

2. INTERFACE FÍSICA

2.1. Conector

A seguir, na Figura 1 podemos observar o padrão de conexão CSKB da placa do projeto NASCERR.

Figura 1 – Interface demandada pelo NASCERR de acordo com conector padrão CSKB.

CubeSat Kit Bus (CSKB)

ESQ-126-39-G-D ou SSQ-126-21-G-D ou TSW-126-07-G-D (parte fêmea voltada para o topo)

H2	2	4	6	8	10	12	14	16	18	20	22	24	26	28	GND 30	GND 32	34	36	38	40	42	44	46	48	/SD_1 50	/SD_2 52
	1	3	5	7	9	11	13	15	17	19	21	23	25	27	GND 29	31	33	35	37	39	41	43	45	47	49	/SD_3 51
H1	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52
	1	3	5	7	9	11	13	15	17	19	I2C1_CLK 21	I2C1_SDA 23	25	27	29	31	33	35	37	39	I2C2_SDA 41	I2C2_CLK 43	45	5V_1 47	5V_2 49	5V_3 51

2.2. Sinais

Tabela 1 – Interface demandada pelo NASCERR de acordo com o conector padrão CSKB.

Pino	Rótulo	Definição
H2.29	GND	Terra.
H2.30	GND	Terra.
H2.32	GND	Terra.
H2.50 - 52	/SD_X[2]	Primeiro, segundo[1] e terceiro[1] sinais de SHUTDOWN.
H1.47	5V_1	Primeiro sinal de 5V.
H1.49	5V_2	Segundo sinal de 5V[1].
H1.51	5V_3	Terceiro sinal de 5V[1].
H1.21	I2C1_CLK	Linha de dados do I2C1[1].
H1.23	I2C1_SDA	Linha de clock do I2C[1].
H1.41	I2C2_SDA	Linha de dados do I2C2.
H1.43	I2C2_CLK	Linha de clock do I2C2.

⚠ ATENÇÃO

- [1]Os sinais de 5V_2, 5V_3, o barramento I2C1 e os sinais /SD_2 e /SD_3 são **comumente desconectados, mas** podem ser conectados quando necessário.
- [2]Os sinais de SHUTDOWN, identificados como /SD_X, são usados para realizar o **desligamento total do sistema**. Quando o sinal de entrada em **/SD_X está em nível HIGH**, todo o sistema do NASCERR é desligado.

2.3. Energização e Inicialização

Para energizar corretamente o payload do NASCERR, a seguinte sequência de eventos deve ser respeitada:

Tabela 2 – Processo de Energização

Seq#	Estado	5V_1	5V_2	5V_3	Espera (ms)	Descrição
0	IDLE	0.0	N/A	N/A	~	Payload desligado
1	NASCERR ON	5.0	N/A	N/A	200.0	Payload energizado

Processo para energização do NASCERR:

- Incialmente o payload está desligado com 0.0 V em seu barramento de alimentação;
- Certificar-se que os sinais de SHUTDOWN estão em nível lógico LOW ou flutuantes;
- Realizar a ligação do barramento de alimentação colocando 5 volts nas entradas 5V_X;
- Aguardar cerca de 200 ms para estabilização.

Inicialização do sistema do NASCERR:

- Inicialmente é executado o sistema de Bootloader;
- Realiza a verificação quanto a pendência de atualização, onde caso seja necessário realiza esse processo de substituição do firmware da aplicação;
- Caso não seja identificada atualizações pendentes realiza a verificação de integridade da aplicação e inicializa a mesma;
- Esse processo pode durar entre 5 e 30 segundos a depender da necessidade de atualização de firmware.

⚠ **ATENÇÃO**

- **Não deverá haver comunicação I2C no payload enquanto estiver em processo de bootloader, iniciando a comunicação I2C apenas quando a aplicação for executada.**

2.4. Desenergização

Para desenergizar corretamente o NASCERR o seguinte procedimento é recomendado:

- Opcionalmente colocar sinal HIGH em /SD_X (/SD_X = 1);
- Desligar linhas permanentes de energia de 5 volts em qualquer ordem.

3. PROTOCOLO DE COMUNICAÇÃO NASCERR

3.1. Introdução

Este documento descreve em detalhes o protocolo de comunicação desenvolvido para a interação entre o OBC e a carga útil do nanossatélite. O protocolo permite a troca de informações de forma robusta e confiável, garantindo que os comandos e dados sejam interpretados corretamente em ambas as extremidades da comunicação.

3.2. Especificações Gerais

O NASCERR atua como um *SLAVE* no barramento I2C, seguindo os seguintes parâmetros:

Tabela 3 – Configurações do SLAVE NASCERR no barramento I2C.

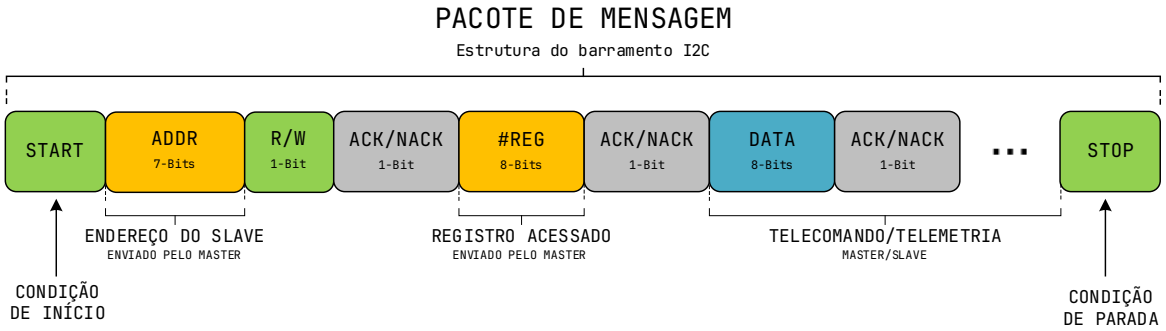
Parâmetro	Valor
Tamanho do Endereço	7-bits
Endereço do Barramento (ADDR)	**0x55**
Frequência de operação (kHz)	100

A interface mostrada a seguir utilizada os seguintes pinos do CDKB:

- **H1-43**: **SCL**: sinalização de clock do barramento I2C.
- **H1-41**: **SDA**: sinalização de dados do barramento I2C.

O protocolo implementado utiliza uma abordagem baseada em mapeamento de registradores, onde uma solicitação de escrita no I2C representa um comando e uma solicitação de leitura representa o envio da resposta, conforme mostrado na Figura 2 abaixo:

Figura 2 - Representação gráfica do pacote no barramento I2C.

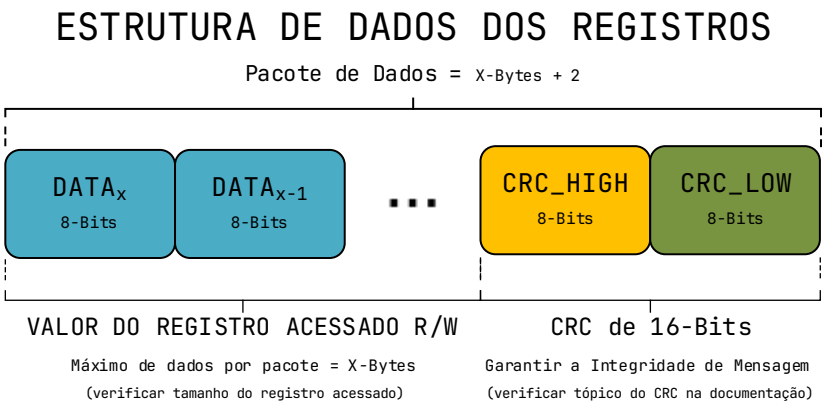


- **START:** Condição de start do I2C;

- **ADDR (7 bits): 0x55** – Endereço da carga útil no I2C;
- **R/W(1-bit):** Bit de solicitação de leitura/escrita;
- **#REG(8 bits):** Registro acessado;
- **ACK/NACK(1 bit):** Bit de ACK/NACK no I2C;
- **DATA [1...n]:** Pacote de dados de Telecomando/Telemetria (8 bits);
- **STOP:** Condição de stop do I2C

Para garantir a integridade dos dados transmitidos pelo barramento I2C, todos os dados enviados e recebidos utilizam um *checksum* de 16 bits (CRC16). O processo de transmissão consiste na anexação do CRC ao final da mensagem, enquanto o receptor calcula o CRC dos dados recebidos e compara com o valor anexado. Caso os valores não coincidam, ocorre uma falha de transmissão.

Figura 3 – Representação gráfica da estrutura do pacote com CRC16.



Cada pacote enviado pelo barramento I2C possui a seguinte estrutura:

1. **Dados**:
   - Representados por **$DATA_x$, $DATA_{(x-1)}$, ...,** com cada segmento contendo 8 bits.
   - O número de segmentos pode variar dependendo da mensagem a ser transmitida.
2. **CRC**:
   - Dividido em duas partes:
     - **CRC_HIGH**: Byte mais significativo do CRC (8 bits).
     - **CRC_LOW**: Byte menos significativo do CRC (8 bits).
   - É calculado utilizando o algoritmo padrão CRC-16 com o polinômio **0xA001**.

3.3. Mapa de Registradores

A NASCERR possui os seguintes registradores acessíveis via I2C:

Tabela 4 – Registradores acessíveis via I2C.

Nome	Reg Addr	R/W	Tamanho(bytes)	Descrição
STAT	0x01	R	13	Registro de status de operação
IDX	0x02	R	4	Registro do número de bytes a serem transmitidos
TXB	0x03	R	2066	Registro com dados de TELEMETRIA indicado por IDX
CMD	0x04	W	31	Registro de TELECOMANDO de experimento
SLEEP	0x06	W	3	Registro do modo SLEEP
WAKEUP	0x07	W	3	Registro do modo WAKEUP
GPS	0x08	W	36	Registro de dados do GPS
HW_STAT	0x0E	W	3	Registro de status de hardware
HW_TXB	0x0F	R	22	Registro de transmissão das informações de hardware
BOOT_DATA	0x10	W	263	Registro de Bootloader
BOOT_STAT	0x11	R	15	Registro de consulta de status de Bootloader
EXP_STAT	0x13	R	18	Registro com sumário de experimentos executados
EXP_SUM_IDX	0x14	W	3	Registro de offset para obtenção de sumário por página de dados de telemetria
EXP_SUM_DATA	0x15	R	216	Registro com sumário com as páginas de experimentos não lidos
DB_CLEAR	0x16	W	3	Registro com comando para apagar estrutura de dados presente no NASCERR
LOG_SUM	0x17	R	16	Registro de Sumário de Logs
LOG_SET_PAGE	0x18	W	3	Registro para selecionar a página de logs a ler
LOG_READ_PAGE	0x19	R	2095	Registro para ler o conteúdo da página com logs selecionada
Tamanho referente ao número de bytes úteis + CRC.				

STAT – Registro de status de operação

Os registros de **STAT** estão organizados em três categorias principais:

- **OP Register (Registro de Operação)**: Identifica o status operacional do sistema.
- **Request Register (Registro de Requisições)**: Gerencia solicitações realizadas ao sistema.
- **Fail Register (Registro de Falhas)**: Identifica as falhas do sistema.

O registro apresenta o **tamanho** de 7 bytes (5 *bytes de dados* + *2 bytes de CRC16*), sendo 2 bytes do OP Register, 2 bytes do Request Register e 1 byte do Fail Register.

OP Register – Registro de Operação

O registro de operação indica o estado operacional atual do sistema. Tem-se duas categorias de registro de operação: Operação do Experimento e Operação do Hardware.

Tabela 5 – Categorias operacionais do experimento.

Nome	Status Code	Descrição
OP_EXP_IDLE	0xA1	Registro para status de AUSÊNCIA DE EXPERIMENTOS
OP_EXP_RUNNING	0xA2	Registro para status de EXPERIMENTO EM EXECUÇÃO
OP_EXP_FAIL	0xA3	Registro para ação na memória finalizada com FALHA

Tabela 6 – Categorias operacionais do hardware.

Nome	Status Code	Descrição
OP_HW_OK	0xB1	Registro que indica que o SISTEMA ESTÁ ACORDADO (IDLE)
OP_HW_SLEEP	0xB2	Registro que indica que o SISTEMA ESTÁ DORMINDO
OP_HW_RX_COMMAND	0xB3	Registro que indica RECEBIMENTO DE COMANDO DE HARDWARE
OP_HW_TX_RESPONSE	0XB4	Registro que indica a PRONTIDÃO DO SISTEMA PARA ENVIO DA RESPOSTA DE HARDWARE
OP_HW_FAIL	0xB5	Registro que indica que houve uma falha no hardware

Uso

OP_EXP_IDLE: Indica que o sistema está sem experimentos em execução, em estado de repouso (IDLE).

OP_EXP_RUNNING: Representa que o sistema está com experimentos em andamento. Entra nesse estado após terminar a escrita na memória.

OP_EXP_FAIL: Indica que houve uma falha durante a execução da operação de escrita ou leitura.

OP_HW_OK: Indica que o sistema está acordado.

OP_HW_SLEEP: Indica que o sistema está dormindo.

OP_HW_RX_COMMAND: Indica que o sistema recebeu um comando de consulta de status do hardware acionado pelo *HW_STAT*.

OP_HW_TX_RESPONSE: Indica que o sistema já processou o comando de consulta de status e está pronto para envio dos dados aquisitados.

OP_HW_FAIL: Indica que o sistema encarou uma superação dos limiares relativos a hardware esperados.

Request Register – Registro de Requisições

O registro de requisição especifica as solicitações feitas ao sistema para aquisição de dados ou transmissão de resultados. Tem-se duas categorias de registro de requisições: Requisições do Experimento e Requisições de Hardware.

Tabela 7 – Status de requisições do experimento.

Nome	Status Code	Descrição
REQ_EXP_NO_REQUEST	0xA1	Sem requisição de experimento para o MASTER
REQ_EXP_TX_RESULT	0xA2	Registro de requisição de TRANSMISSÃO DO EXPERIMENTO CONCLUÍDO

Tabela 8 – Status de requisições do HARDWARE.

Nome	Status Code	Descrição
REQ_HW_NO_REQUEST	0xB1	Sem requisição de hardware para o MASTER
REQ_HW_TX_TEMP	0xB2	Registro de Transmissão de informação da temperatura
REQ_HW_TX_CURRENTS	0xB3	Registro de Transmissão de informação das correntes individualmente do hardware
REQ_HW_TX_POWERS	0xB4	Registro de Transmissão de informação das potências individualmente do hardware
REQ_HW_TX_TOT_PWR	0XB5	Registro de Transmissão de informação da potência geral do hardware

Uso

- **REQ_EXP_NO_REQUEST:** Utilizado como valor padrão para indicar que nenhuma solicitação a respeito do resultado experimento está ativa.
- **REQ_EXP_TX_RESULT**: Solicita que os resultados do experimento sejam transmitidos. Armado quando o Registro *TXB* é preenchido.
- **REQ_HW_NO_REQUEST**: Utilizado como valor padrão para indicar que nenhuma solicitação de hardware está ativa.
- **REQ_HW_TX_TEMP**: Solicita que os dados referentes a temperatura do sistema sejam transmitidos. Armado quando enviado solicitado pelo MASTER via *HW_STAT*.
- **REQ_HW_TX_CURRENTS**: Solicita que os dados referentes as correntes do sistema sejam transmitidas. Armado quando enviado solicitado pelo MASTER via *HW_STAT*.
- **REQ_HW_TX_POWERS**: Solicita que os dados referentes as potências do sistema sejam transmitidas. Armado quando enviado solicitado pelo MASTER via *HW_STAT*.
- **REQ_HW_TX_TOT_PWR**: Solicita que o dado referente a potência total do sistema seja transmitido. Armado quando enviado solicitado pelo MASTER via *HW_STAT*.

Fail Register – Registro de Falhas

O registro de falha especifica os problemas apresentados pelo sistema. As falhas são referentes ao hardware (*FAIL_HW*).

Tabela 9 – Falhas de Hardware do sistema.

Nome	Bit	Descrição
Current_1V2	0	Indica se o limiar foi de corrente 1V2 estimado foi superado
Current_2V5	1	Indica se o limiar foi de corrente 2V5 estimado foi superado
Current_2V5VPP	2	Indica se o limiar foi de corrente 2V5_VPP estimado foi superado
Current_3V3	3	Indica se o limiar foi de corrente 3V3 estimado foi superado
Current_3v3_FPGA	4	Indica se o limiar foi de corrente 3V3_FPGA estimado foi superado
Temperature	5	Indica se o limiar foi de Temperatura estimado foi superado

Uso

Utilização:

O sistema periodicamente verifica parâmetros do hardware. Quando a medição supera o limiar operacional esperado a flag condizente é armada. A Flag apresenta o t**amanho de 1 byte.**

- **Exemplo:**

  Se o valor de corrente medido na 1V2 ficar maior que o limiar por um número de 16 amostragens ou mais o bit 0 (Current_1V2) é armado.

IDX – Registro do Número de Bytes a Serem Transmitidos

O registro IDX indica o número total de bytes de dados de telemetria que precisam ser transmitidos no pacote.

Uso

- **Utilização:**
  - Este registro é lido pelo sistema para determinar o tamanho total da transmissão de telemetria. **Tamanho:** 2 bytes *(2 bytes de dados + 2 bytes de CRC16).*
  - Os 4 bytes do registro são enviados na ordem do mais significativo para o menos significativo (big-endian).
- **Exemplo:**
  - Se o valor armazenado no **IDX** for **0x00001000** (4 bytes), isso indica que o sistema deve transmitir **4096 bytes** de telemetria.

TXB – Registro de Dados de Telemetria

O registro onde os dados de telemetria são armazenados para serem transmitidos.

Uso

- **Função:** Disponibilizar ao host os resultados de experimento(s) em blocos de 88 bytes por item. **Tamanho:** variável *(N × 88 bytes + 2 bytes de CRC16).*
- **Utilização:**
  - Ler **IDX** → obter **exp_len**.
  - Ler **TXB** com exatamente **exp_len** bytes.
  - Validar CRC16 de **TXB**; remover os 2 bytes finais (CRC).
  - Fatiar o *payload* em blocos de 88 bytes e desserializar **NASCERR_EXPERIMENT**.
- **Exemplo:**
  - Se **IDX = 0x00B2** (178), então o *payload* de **TXB** tem **176 bytes = 2 itens** (2 × 88) e mais **2 bytes** de CRC16.
- **NOTA: A ORGANIZAÇÃO E OS DETALHES DOS DADOS DE TELEMETRIA ESTÃO DESCRITOS NO TÓPICO DEDICADO À TELEMETRIA.**

CMD – Registro de Telecomando do Experimento

Esse registro recebe comandos externos para passar parâmetros e agendar a execução de experimentos do **NASCERR**.

Uso

- **Função:** Centralizar o recebimento e processamento de comandos externos, permitindo configurar parâmetros e iniciar experimentos no NASCERR. **Tamanho:** 31 bytes *(29 bytes de dados + 2 bytes de CRC16).*
- **Utilização:**
  - O sistema escreve no registro CMD e o NASCERR processa esses comandos recebidos e realizar as operações solicitadas.
  - Permite que o sistema configure e inicie experimentos pré-definidos, ajustando parâmetros de execução.
- **Exemplo:**
  - Um comando pode configurar:
    - A quantidade de dados a serem escritos na memória.
    - O tipo de algoritmo de correção de erro a ser utilizado.
    - A quantidade de repetições para as operações de escrita e leitura.

- **NOTA: A ESTRUTURA DETALHADA DOS COMANDOS DE TELECOMANDO, INCLUINDO SEUS PARÂMETROS E FUNCIONALIDADES, É APRESENTADA NO TÓPICO DEDICADO AO TELECOMANDO.**

SLEEP – Registro para Entrada no Modo de Economia de Energia

Registro que configura o NASCERR para entrar no modo de economia de energia (SLEEP), mediante a escrita de um valor específico.

Uso

- **Função:** O NASCERR entra no modo de economia de energia somente quando o valor **0xAA** é escrito no registro **SLEEP**. **Tamanho:** 3 bytes *(1 bytes de dados + 2 bytes de CRC16).*
- **Utilização:**
  - Escrever qualquer valor diferente de **0xAA** não terá efeito, garantindo maior confiabilidade contra erros ou ativações não intencionais.
  - No modo **SLEEP**, o sistema desativa componentes não essenciais para reduzir o consumo energético.
- **Exemplo:**
  - O sistema escreve o valor **0xAA** no registro **SLEEP** para entrar no modo de economia de energia.
  - **Valor Inválido**: Escrever outro valor, como **0x00**, será ignorado pelo sistema.

WAKEUP – Registro para Saída do Modo de Economia de Energia

Registro que configura o NASCERR para sair do modo de economia de energia (WAKEUP), mediante a escrita de um valor específico.

Uso

- **Função:** O NASCERR sai do modo de economia de energia somente quando o valor **0xAA** é escrito no registro **WAKEUP**. **Tamanho:** 3 bytes *(1 bytes de dados + 2 bytes de CRC16).*
- **Utilização:**
  - Escrever qualquer valor diferente de **0xAA** não terá efeito, garantindo maior confiabilidade contra ativações não autorizadas ou acidentais.

- Ao sair do modo **SLEEP**, o sistema reativa os componentes necessários para operação normal.

- **Exemplo:**
  - O sistema escreve o valor **0xAA** no registro **WAKEUP** para sair do modo de economia de energia.
  - **Valor Inválido:** Escrever outro valor, como **0xFF**, será ignorado.

GPS – Registro de Dados GNSS e Localização

Registro que armazena informações de tempo GNSS e coordenadas de localização, atualizado periodicamente para ser anexado às respostas de experimento.

Tabela 10 – Estrutura que representa o tempo GNSS e a localização fornecida pelo MDCA.

Campo	Tipo	Tamanho (bytes)	Descrição
gnss_week	U16	2	Número da semana GNSS
gnss_tow	U32	4	Tempo da semana GNSS em segundos
latitude	F32	4	Latitude em graus decimais
longitude	F32	4	Longitude em graus decimais
altitude_ellipsoid	F32	4	Altitude em relação ao elipsoide de referência
altitude_msl	F32	4	Altitude em relação ao nível médio do mar (MSL)
ecef_x	F32	4	Coordenada X no sistema ECEF
ecef_y	F32	4	Coordenada Y no sistema ECEF
ecef_z	F32	4	Coordenada Z no sistema ECEF

Uso

- **Função:** Fornecer contexto de tempo e posição geográfica em cada resultado de experimento transmitido pelo NASCERR. **Tamanho:** 36 bytes *(34 bytes de dados + 2 bytes de CRC16)*.
- **Utilização:**
  - Atualizado automaticamente em períodos definidos (ex.: 1 Hz) pelo sistema.
  - Os dados são anexados ao *payload* de resposta sempre que um experimento é concluído e transmitido.
- **Exemplo:**
  - O NASCERR coleta dados GNSS atualizados, preenche o registro interno e insere automaticamente esses 34 bytes no final no pacote de telemetria do experimento lido.
  - Caso não haja dados válidos, os campos permanecem zerados.

HW_STAT – Registro de Status de Hardware

O registro HW_STAT é responsável por realizar o processo interpretação dos dados de hardware requisitados e o preenchimento destes no buffer de transmissão (*HW_TXB*). A requisição do master é feito de acordo com os dados da Tabela *11*.

Tabela 11 – Requisições de dados do hardware.

Campo	Valor	Descrição
TEMP_DATA	0xB1	Requisição de temperatura atual do sistema
CURRENT_DATA	0XB2	Requisição dos parâmetros de corrente do sistema. (1V2, 2V5, 2V5_VPP, 3V3 e 3V3_FPGA)
POWER_DATA	0XB3	Requisição de parâmetros de potência instantânea do sistema. (1V2, 2V5, 2V5_VPP, 3V3 e 3V3_FPGA)
POWER_CONSUMED	0XB4	Requisição dos parâmetros de potência total instantânea do sistema.

Uso

- **Função:** Indicar e preencher o parâmetro a ser enviado pelo buffer de transmissão de hardware. **Tamanho:** 3 bytes *(1 bytes de dados + 2 bytes de CRC16)*.
- **Utilização:** MASTER escreve a informação que deseja do hardware. Os dados requisitados moldam o tamanho do buffer de transmissão. Dados de temperatura e potência total enchem o buffer de transmissão com *4 bytes de dados + 2 bytes de CRC16*, já os dados de potência e corrente instantânea com *20 bytes de dados + 2 bytes de CRC16*.
- **Exemplo:**
  - O NASCERR recebe do master o comando **TEMP_DATA,** coleta dados de temperatura atualizados e insere automaticamente esses dados no buffer de transmissão de hardware.
  - **Envio de valor não esperado**: Um *warning* é gerado e a última configuração prevalece.

HW_TXB – Registro de Dados de Hardware

O registro HW_TXB é responsável por realizar o processo de envio dos dados de hardware requisitados. A forma dos dados a serem transmitidos estão representadas na Tabela *12*.

Tabela 12 – Informações disponíveis do hardware.

Campo	Tipo	Tamanho (bytes)	Descrição
Temperatura	F32	4	Temperatura do sistema

Correntes	F32	20	Correntes referentes as medições dos pontos de tensão 1V2, 2V5, 2V5_VPP, 3V3 e 3V3_FPGA do sistema.
Potências	F32	20	Potências referentes as medições dos pontos de tensão 1V2, 2V5, 2V5_VPP, 3V3 e 3V3_FPGA do sistema.
Potência total consumida	F32	4	Potência total instantânea do sistema.

Uso

- **Função:** O registro envia os dados previamente requisitados. **Tamanho:** 6 bytes *(4 bytes de dados + 2 bytes de CRC16)* caso os dados previamente requisitados fossem de temperatura ou potência total consumida ou 22 bytes *(20 bytes de dados + 2 bytes de CRC16)* caso os dados requisitados sejam de correntes e potências.
- **Utilização:**
  - O MASTER solicita os dados de acordo com o tamanho esperado pela requisição previamente realizada e há esta transmissão.
  - Os dados sempre são referentes a última requisição e armazenam o valor observado no momento da requisição.
- **Exemplo:**
  - O sistema acessa o registro e os dados de temperatura previamente requisitados são enviados.

BOOT_DATA – Registro Transmissão de Dados para Atualização de Firmware

O registro BOOT_DATA é responsável por realizar o processo de recepção dos dados do novo firmware, ele possui um padrão de formato de dados para realizar o gerenciamento desse processo. O registro espera mensagens com a estrutura definida na Tabela *13*, de modo que os campos do *payload* são definidos de acordo com o tipo de comando enviado.

Tabela 13 – Estrutura de Registro de Transmissão de dados de Bootloader.

Campo	Tipo	Tamanho (bytes)	Descrição
bytes_msg	U16	**2**	Campo que indica o tamanho da mensagem
cmd_type	U8	**1**	Comandos de bootloader
payload	-	**0 ~ 258**	*Payload* de dados que muda de formato a depender do comando enviado
crc16	U16	**2**	*Checksum* para verificar integridade do pacote

Os comandos que podem ser transmitidos ao registro de BOOT_DATA são os definidos na Tabela *14*.

Tabela 14 – Comandos esperados pelo registro de BOOT_DATA.

Comando	Valor	Descrição

DOWNLOAD_INIT	**0x01**	Comando que prepara NASCERR para iniciar transferência de dados
DATA_TRANSFER	**0xAA**	Comando de transferência de dados com o novo binário
DONWLOAD_FINISH	**0x02**	Comando de finalização de *donwload* e autorização para instalar o firmware recebido

Para cada tipo de comando que pode ser transmitido ao registro de BOOT_DATA o *payload* que foi definido na Tabela *13* varia seus campos de acordo com as Tabela *15*, Tabela *16* e Tabela *17*.

Tabela 15 – Formato do *payload* para comando DOWNLOAD_INIT.

Campo	Tipo	Tamanho (bytes)	Descrição
firmware_size	U32	**4**	Campo que indica o tamanho do binário que será transmitido
version	U32	**4**	Campo que indica a versão do novo binário que está sendo transmitido
num_packets	U16	**2**	Campo que indica a quantidade de pacotes que serão transmitidos no processo de envio de *firmware*

Tabela 16 – Formato do *payload* para comando DATA_TRANSFER.

Campo	Tipo	Tamanho (bytes)	Descrição
packet_id	U16	**2**	Campo que indica o ID do pacote que está sendo transmitido
chunk	-	**0 ~ 256**	Pacote de dados binários

Tabela 17 – Formato do *payload* para comando DOWNLOAD_FINISH.

Campo	Tipo	Tamanho (bytes)	Descrição
firmware_version	U32	**4**	Campo que indica a versão do *firmware* que acabou de ser transmitida
sha_256	-	**32**	*Hash* de segurança calculado para o *firmware* completo

Uso

- **Função:** Recepcionar os dados relacionados ao processo de atualização de firmware.
- **Utilização: O** MASTER escreve a informação que deseja transmitir ao NASCERR seguindo o padrão do protocolo discutido no decorrer desse registro. **Tamanho:** 263 bytes *(261 bytes de dados + 2 bytes de CRC16).*
- **Exemplo:**
  - O NASCERR recebe do master o comando **de DOWNLAOD_INIT** e prepara o sistema para a recepção do novo firmware.

NOTA: Para mais detalhes consulte a sessão dedicada a explicar o fluxo de atualização.

BOOT_STAT – Registro de Status de Bootloader

O registro BOOT_STAT indica o status do processo de bootloader, podendo-se verificar a versão de firmware que está em execução, e várias outras informações acerca do estado do sistema, tanto no quesito de atualização, quanto ao andamento do processo de atualização de firmware, a Tabela *18* define a estrutura desse registro.

Tabela 18 – Estrutura de Registro de Status de Bootloader.

Campo	Tipo	Tamanho (bytes)	Descrição
current_version	U32	**4**	Versão atual do firmware
new_version	U32	**4**	Nova versão do firmware armazenada
boot_flags	U8	**1**	Flags de bootloader (Tabela *20*)
last_error	U8	**1**	Último erro ocorrido
last_pkg_id	U16	**2**	ID do último pacote de dados recebido
current_app	U8	**1**	Região de atual execução do firmware (Tabela *19*)
crc16	U16	**2**	*Checksum* para verificar integridade do pacote

O registo conta com campos de versão de firmware, o em execução e o que está sendo ou já foi recebido. Um campo que indica o último erro ocorrido no processo, o campo com o ID do último pacote de firmware recebido, e um campo indicando em qual banco de memória o firmware atual está em execução, representada na Tabela *19*. Também há um campo com flags que representam o estado do sistema de atualização, as quais são definidas na Tabela *20*.

Tabela 19 – Estrutura de define regiões em que o firmware está em execução.

Nome	Valor	Descrição
BANK_1	**0xAA**	Banco de memória 1 com a aplicação em execução
BANK_2	**0xBB**	Banco de memória 2 com a aplicação em execução

Tabela 20 – Flags presentes na estrutura de status de bootloader.

Flag	Máscara de bits	Descrição
UPDATE_PENDING	**0x01**	Atualização pendente a ser iniciada ou completada
CRC_VALID	**0x02**	*CRC* do pacote recebido é válido
IMAGE_VALID	**0x04**	Imagem completamente válida e pronta para ser executada
DONWLOAD_SUCCESS	**0x08**	*Donwload* completo sem erros detectados
RETRANSMIT_REQ	**0x10**	Solicitação de retransmissão de pacote
ROLLBACK_AVALIABLE	**0x20**	Imagem válida anterior está pronta para *rollback* caso necessário
ERROR_DETECTED	**0x40**	Erro durante o processo de atualização
IN_BOOTLOADER	**0x80**	Sistema está em modo bootloader e aguarda atualização de *firmware*

Uso

- **Função:** Informar ao MASTER o status do processo de atualização de firmware.
- **Utilização: O** MASTER realiza a leitura do registro para conseguir verificar de firmware em execução no NASCERR, quantos pacotes de firmware já foram recebidos com sucesso, se ocorreu algum erro no processo de recepção ou atualização, entre outras informações. **Tamanho:** 15 bytes *(13 bytes de dados + 2 bytes de CRC16).*
- **Exemplo:**
    - O MASTER consulta este registro e verifica a versão do firmware em execução no NASCERR.

NOTA: Para mais detalhes consulte a sessão dedicada a explicar o fluxo de atualização.

EXP_STAT– Registro de Sumário de Experimentos Executados

Esse registro armazena um resumo do estado geral dos experimentos e agendamentos do NASCERR, permitindo monitorar a fila de execução e a ocupação da memória.

Tabela 21 – Estrutura que representa o status geral de experimentos.

Campo	Tipo	Tamanho (bytes)	Descrição
qtd_scheduler_total	U16	2	Total de agendamentos registrados
qtd_scheduler_pending	U16	2	Agendamentos ainda não executados
qtd_scheduler_executed	U16	2	Agendamentos já executados
qtd_experiments_total	U16	2	Total de experimentos registrados
qtd_experiments_running	U16	2	Experimentos em execução
qtd_experiments_finished	U16	2	Experimentos finalizados
qtd_experiments_unread	U16	2	Experimentos não lidos
scheduler_full_flag	U8	1	Flag de escalonador cheio
flash_full_flag	U8	1	Flag de memória flash cheia

Uso

- **Função:** Resumir o status geral de agendamentos e experimentos do sistema. **Tamanho:** 18 bytes *(16 bytes de dados + 2 bytes de CRC16).*
- **Utilização:** Consultado para estatísticas e flags de capacidade.
- **Exemplo:** Leitura após múltiplos experimentos para obter panorama geral.

EXP_SUM_IDX – Registro de Índice de Páginas de Sumário

Define o offset de página para consulta de sumários no espaço de telemetria.

Uso

- **Função:** Selecionar a página do sumário que será retornada em EXP_SUM_DATA. No bando de dados foram reservados 3 blocos de memória com 64 páginas no total, o que representa 192 páginas de experimentos. **Tamanho:** 3 bytes *(1 bytes de dados + 2 bytes de CRC16).*
- **Utilização:** MASTER escreve o número da página desejada neste registro.
- **Exemplo:** *page_offset* = 0x02 prepara a 3ª página de sumário para leitura em EXP_SUM_DATA.

EXP_SUM_DATA – Registro de Dados de Sumário de Experimentos

Esse registro armazena um bloco de sumário de páginas de experimentos ainda não lidos. É atualizado de acordo com o índice configurado em EXP_SUM_IDX.

Tabela 22 – Estrutura que representa o status geral de experimentos.

Campo	Tipo	Tamanho (bytes)	Descrição
qtd_unread_total	U16	2	Quantidade TOTAL de experimentos não lidos no sistema
offset	U8	1	Offset do **primeiro item** desta página (índice global base)
qtd_included	U8	1	Quantidade de itens **incluídos** nesta página (≤ 30)
entries	**DBItem[30]**	210	Vetor com até **30** entradas compactadas (layout abaixo)

Tabela 23 – Estrutura que representa o DBItem.

Campo	Tipo	Tamanho (bytes)	Descrição
id	U32	4	Identificador do banco
num_block	U16	2	Número de blocos de resposta contido nessa página
status	U8	1	Valor de DBStatus preservado (ver Tabela *24*)

Tabela 24 – Estrutura de DBStatus (valores predefinidos)

Nome	Valor	Descrição
DB_EMPTY	**0x00**	Vaga vazia / sem dados
DB_NOT_READ	**0x01**	Experimento registrado e **ainda não lido**
DB_READ	**0x02**	Experimento **já lido**

DB_CANCELED	**0x03**	Registro cancelado/invalidado
DB_NEW_COMMAND	**0x04**	Novo comando pendente
DB_EXECUTED	**0x05**	Experimento **executado**

Uso

- **Função:** Fornecer um resumo paginado dos experimentos não lidos. **Tamanho:** 216 bytes *(214 bytes de dados + 2 bytes de CRC16).*
- **Utilização:**
  - Escreva o índice desejado em **EXP_SUM_IDX**; em seguida **leia EXP_SUM_DATA** para obter a página correspondente.
  - Use **qtd_included** para saber quantos itens vieram (≤ 30) e **qtd_unread_total** para calcular páginas restantes.
  - O campo **offset** confirma o índice global do **primeiro** item retornado nesta página.
- **Exemplo:**
  - Com 87 itens não lidos, escreva EXP_SUM_IDX = 60 → leitura retorna **qtd_included** = 27 (itens 60...86), **qtd_unread_total** = 87, offset = 60.

DB_CLEAR – Registro de Apagamento de Estruturas de Dados

Esse registro apaga a regiões de armazenamento de informações da memória flash externa do NASCERR, sempre preservando o cabeçalho (FLASH_HEADER).

Tabela 25 – Estrutura que representa o status geral de experimentos.

Campo	Valor	Permissão	Descrição
FLASH_HEADER	**0x00**	⦸	Cabeçalho do DB — **nunca é apagado** por este comando
FLASH_LOGS	**0x01**	☑	Região de logs
FLASH_SCHEDULER	**0x02**	☑	Região do escalonador
FLASH_EXPERIMENT	**0x03**	☑	Região de dados de experimentos
FLASH_BACKUP_FIRMWARE	**0x04**	☑	Região de *backup* de firmware

Uso

- **Função:** Solicitar **um *clear* em uma região** de memória. **Tamanho:** 3 bytes *(1 bytes de dados + 3 bytes de CRC16).*
- **Utilização:** Escrever o código da região e enviar com **CRC16** válido; acompanhar via registros de **STATUS (OP/*REQ*)**.
- **Exemplo:**

- Apagar experimentos executados: Payload 0x03 → Pacote 03 | CRC16_MSB CRC16_LSB.

- **OBSERVAÇÕES: OPERAÇÃO IRREVERSÍVEL; BLOQUEAR/NEGAR SE O ALVO FOR FLASH_HEADER (GERAR *WARNING* E SINALIZAR FALHA). O TEMPO DE EXECUÇÃO DEPENDE DO TAMANHO DA REGIÃO.**

LOG_SUM – Registro de sumário de logs

Esse registro armazena um resumo do estado atual da memória de logs e da página selecionada, permitindo ao Master conhecer rapidamente quantas páginas existem, qual está ativa e o status associado.

Tabela 26 – Estrutura que representa o sumário de logs.

Campo	Tipo	Tamanho (bytes)	Descrição
data_load	U32	4	Quantidade de bytes já gravados na região de logs.
page_with_logs	U16	2	Total de páginas de log disponíveis (qtd_logs).
selected_page	U8	1	Página atualmente selecionada em LOG_SET_PAGE.
hdr.id	U32	4	Identificador salvo no cabeçalho da página selecionada.
hdr.num_block	U16	2	Número de blocos ou registros armazenados nessa página.
hdr.status	U8	1	Valor de DBStatus preservado (ver Tabela *24*)

Uso:

- **Função**: Fornecer metadados resumidos da região de logs e da página selecionada. **Tamanho**: 16 bytes *(14 bytes de dados + 2 bytes de CRC16).*
- **Utilização**: Consultado após seleção de uma página com **LOG_SET_PAGE** para conhecer status e metadados da mesma.

Observação: Sempre atualizado quando o comando **LOG_SET_PAGE** é processado.

LOG_SET_PAGE – Registro para selecionar a página de logs a ler

Esse registro define qual página da região de logs será preparada para leitura. Ele é análogo ao EXP_SUM_IDX usado para experimentos.

Uso:

- **Função**: Selecionar a página de logs que será processada para retorno em **LOG_SUM** e **LOG_READ_PAGE**. **Tamanho**: 3 bytes *(1 byte de dados + 2 bytes de CRC16)*.
- **Utilização**: O Master escreve o número da página desejada nesse registrador. O firmware então atualiza **LOG_SUM** e **LOG_READ_PAGE** conforme a seleção.
- **Exemplo**: **LOG_SET_PAGE = 0x04** → prepara a 5ª página de logs para leitura.

LOG_READ_PAGE – Registro para ler o conteúdo da página com logs selecionada

Esse registro armazena os registros compactados de uma página de logs, conforme definido em **LOG_SET_PAGE**. Cada entrada ocupa 7 bytes.

Tabela 27 – Estrutura de cada entrada de log.

Campo	Tipo	Tamanho (bytes)	Descrição
timestamp	U32	4	Marca de tempo de log.
type	U8	1	Tipo de log.
status	U8	1	Status de log.
error_code	U8	1	Código de erro associado.

Tabela 28 – Código de erro associado de logs.

Nome	Código	Descrição
ERR_NONE	0x01	Sem erro
WARN_COMM_DECODE	0x02	Falha na decodificação do comando
WARM_HW_STATUS_CODE	0x03	Tentativa de acesso de status de hardware inexistente
ERR_COMM_SENDPKT	0x04	Falha no envio do pacote
ERR_EXP_INVALID_DATALEN	0x05	Experimento interrompido: dado de tamanho inválido
ERR_EXP_MEM_OVERFLOW	0x06	Experimento interrompido: estouro de região de memória
ERR_EXP_INVALID_REPETITIONS	0x07	Experimento interrompido: número de repetições inválido
ERR_EXP_INVALID_DELAY	0x08	Experimento interrompido: valor de delay inválido
ERR_EXP_CONFLICT	0x09	Experimento em conflito
ERR_EXP_SCH_FULL	0x0A	Agendador cheio
ERR_EXP_READ_OUT_OF_RANGE	0x0B	Tentativa de leitura fora do intervalo de experimentos
ERR_I2C_INIT_FAILED	0x0C	Erro na inicialização da comunicação I²C

ERR_I2C_BUSY	0x0D	Comunicação I²C ocupada
ERR_I2C_TRANSFER	0x0E	Erro na transmissão/recepção I²C
ERR_I2C_TIMEOUT	0x0F	Transmissão finalizada por atingir timeout
ERR_FLASH_OPERATION	0x10	Erro de operação na flash
ERR_FLASH_ERASE_FAILED	0x11	Erro ao apagar bloco na flash
ERR_PROTO_CMD_DECODE	0x12	Falha na decodificação do comando
ERR_PROTO_SEND_PACKET	0x13	Falha no envio do pacote
ERR_COMM_GENERAL	0x14	Erro de comunicação genérico
ERR_CMD_INVALID	0x15	Comando inválido ou desconhecido
ERR_CMD_OVERFLOW	0x16	Estouro na fila de comando
ERR_CMD_CRC	0x17	Erro no CRC do comando
ERR_DB_INIT	0x18	Erro de inicialização do banco de dados
ERR_DB_WRITE	0x19	Erro de escrita no banco de dados
ERR_BOOT_CMD_SIZE	0x1A	Erro de tamanho de comando do bootloader
ERR_BOOT_CMD_INIT	0x1B	Erro de inicialização de comando do bootloader
ERR_BOOT_CMD_DECODE	0x1C	Erro de decodificação de comando do bootloader
ERR_BOOT_FLASH_WRITE	0x1D	Erro de escrita na flash do bootloader
ERR_BOOT_FLASH_ERASE	0x1E	Erro de apagamento na flash do bootloader
ERR_BOOT_FLASH_WRITE_SIZE	0x1F	Erro de tamanho de escrita na flash do bootloader
ERR_BOOT_FLASH_CRC	0x20	Erro de CRC na flash do bootloader
ERR_DB_READ	0x21	Erro de leitura no banco de dados
ERR_BOOT_PKG_OUT_OF_SEQ	0x22	Erro de pacote fora de sequencia no bootloader
ERR_BOOT_PKG_INVALID_VER	0x23	Erro de versão invalida no bootloader
ERR_BOOT_TIMEOUT	0x24	Timeout no bootloader

Tabela 29 – Códigos de tipo de logs.

Nome	Código	Descrição
LOG_SUCCESS	0x01	Operação bem-sucedida
LOG_RUNNING_EXPERIMENT	0x02	Experimento em execução
LOG_RECEIVE_COMMAND	0x03	Comando recebido e em processamento
LOG_RECEIVE_HW_STAT_REQ	0x04	Comando de recebimento de algum aspecto do hardware
LOG_TRANSMIT_HW_RESPONSE	0x05	Resposta do hardware pronta para ser transmitida
LOG_TRANSMIT_HW_FINISH	0x06	Comando de hardware transmitido

LOG_HW_WARN	0x07	Aviso de hardware
LOG_HW_ERROR	0x08	Erro de hardware
LOG_TRANSMIT_RESPONSE	0x09	Resposta em transmissão
LOG_TRANSMIT_FINISH	0x0A	Transmissão finalizada com sucesso
LOG_EXP_EXECUT	0x0B	Experimento em execução
LOG_EXP_FINNISH	0x0C	Experimento finalizado
LOG_SYSTEM_BUSY	0x0D	Sistema ocupado, não pode aceitar novo comando
LOG_FINISH	0x0E	Operação finalizada (status genérico)
LOG_FAIL	0x0F	Operação falhou (erro genérico)
LOG_MAX	0x10	Valor máximo
LOG_BOOT_INITIALIZED	0x11	Bootloader inicializado
LOG_BOOT_RECEIVE_COMMAND	0x12	Comando de bootloader recebido
LOG_BOOT_OPERATION	0x13	Operação de bootloader em andamento

Uso:

- **Função**: Fornecer ao Master os registros compactados da página atual de logs. **Tamanho:** variável *(N × 7 bytes + 2 bytes de CRC16)*.
- **Utilização**: Após selecionar página com **LOG_SET_PAGE**, ler **LOG_READ_PAGE** para receber os logs daquela página. O Master deve iterar de 7 em 7 bytes até **qtd_included** informado no **LOG_SUM**.
- **Exemplo**: Com página 2 selecionada, leitura retorna 32 entradas (cada com 7 bytes), totalizando 224 bytes de *payload* + 2 de CRC16.

Observação:

- O CRC16 é calculado sempre sobre o *payload* (todos os logs empacotados).
- Após uma leitura bem-sucedida, a página é marcada como **DB_READ**, e essa mudança aparecerá no próximo **LOG_SUM**.

3.4. Telecomandos do NASCERR

O **telecomando** é a interface de **comando e controle** entre o sistema mestre (OBC/host) e o NASCERR. Por meio dele, o mestre solicita **leituras** de informações/telemetria ou **agenda** a execução de experimentos. Os tipos de argumentos previstos são descritos na Tabela *30*.

Tabela 30 – Tipos de comando (cmd_type).

Tipo	Valor	Descrição
READ_CMD	**0x00**	Leitura de registros/telemetria/resultado (não altera estado).
WRITE_CMD	**0x01**	Específica/agenda uma escrita/experimento.
Demais códigos: reservados e devem ser rejeitados.		

Formato do Telecomando

Estrutura base do frame transmitido para o registrador CMD:

Tabela 31 – Estrutura do telecomando (NASCERR_CMD).

Campo	Tipo	Tamanho (bytes)	Descrição
bytes_msg	U16	2	Número total de bytes na mensagem
cmd_type	U8	1	Tipo do comando (operação de leitura ou escrita)
scheduler	nascerr_scheduler	24	Dados do agendador para o experimento
crc16	U16	2	CRC16 checksum

Tabela 32 – Estrutura do campo de nascer_scheduler.

Campo	Tipo	Tamanho (bytes)	Descrição
id	U32	4	Identificador da mensagem/experimento.
block	U16	2	Bloco de dados (0…63). Na leitura, define a página exposta em EXP_SUM_DATA.
qtd_block	U16	2	Quantidade de blocos (0…63). Na leitura, define quantas estruturas de telemetria serão enviadas.
ecc	U8	1	Modo ECC: 00-Indef.; 01-Bypass SRAM1; 02-Bypass SRAM2; 03-TBEC; 04-MRSC; 05-Bypass duplo.
write_type	U8	1	Padrão de escrita: 00→0x00; 01→0x11; 02→0x55; 03→0xAA.
scheduler_time	U32	4	Horário programado para execução (delay em ms).
repeat	U16	2	Número de repetições programadas
delay	U32	4	Intervalo entre repetições, em ms
current_tick	U32	4	Tick atual do sistema operacional (deixar em branco – setado internamente)
Tamanho: 29 bytes *(27 bytes de dados + 2 bytes de CRC16).*			

Escrita/Agendamento (WRITE_CMD = 0x01)

- ***block*** = bloco inicial de escrita; ***qtd_block*** = quantidade de blocos a escrever.
- ***ecc***, ***write_type***, ***scheduler_time***, ***repeat***, ***delay*** são **ativos** e determinam o perfil do experimento.
- ***current_tick*** é preenchido internamente pelo NASCERR.

Leitura (READ_CMD = 0x00)

Na operação de leitura, apenas parte do *scheduler* é interpretada:

- ***block:*** seleciona a página do banco de dados que será lida, que pode ser vista pela consulta no registrador **EXP_SUM_DATA**.
- ***qtd_block:*** define quantas estruturas de telemetria o NASCERR deve preparar para envio sequencial via **TXB**, a partir do *block* informado, respeitando:
    - o limite indicado pelo campo ***num_block*** em **EXP_SUM_DATA** (quanto há/foi preparado para essa consulta);
    - o limite do firmware de 23 estruturas por consulta (teto operacional).
- Demais campos do *scheduler* são ignorados em **READ_CMD**.

Fluxo típico de leitura

1. Mestre envia NASCERR_CMD com **cmd_type=READ_CMD**, **block=*X***, **qtd_block=*Y*** (demais campos zerados).
    - **cmd_type = READ_CMD (0x00)**
    - **block = *X*** → define **qual página** da memória será transmitida por **TXB**.
    - **qtd_block = Y** → define **quantas estruturas de telemetria** o NASCERR deve transmitir por **TXB** (***máximo MAX_EXPERIMENTS_PER_PAGE = 23 por consulta***).
2. NASCERR valida tamanho/limites/CRC e, se ok, preenche o registrador de leitura **TXB** e atualizar **STATUS/EXP_STAT (no tópico de telemetria é apresentado com mais detalha o processo leitura)**.

Registradores relacionados à leitura:

- **EXP_SUM_DATA**: janela de dados/telemetria da página corrente.

- **EXP_SUM_IDX:** índice da página atual (o firmware pode atualizá-lo internamente ao processar block).
- **TXB:** registrador de leitura por onde as estruturas de telemetria são efetivamente transmitidas ao host.

Exemplo (READ_CMD):

bytes_msg=29, **cmd_type**=0x00, **scheduler.block**=10, **scheduler.qtd_block**=3, demais campos **0x00; crc16**=(calculado).

Com isso, o dispositivo prepara até **3 estruturas** contida na **página 10** da memória de experimentos lidos, limitado por ***num_block*** e pelo teto ***MAX_EXPERIMENTS_PER_PAGE***.

3.5. Telemetria NASCERR

Após a realização do experimento o **NASCERR** irá retornar como resposta os dados de telemetria resultantes do experimento, esses dados ficam armazenados no registro **TXB**, de onde o OBC deve realizar a leitura seguindo o fluxo de telemetria.

Formato da Telemetria

Item que representa um resultado de experimento, composto por cabeçalho, ECC_Telemetry (30 bytes) e MDCA (34 bytes).

Tabela 33 – Estrutura de dados do NASCERR_EXPERIMENT.

Campo	Tipo	Tamanho (bytes)	Descrição
id	U32	4	Identificador da mensagem
block	U16	2	Identificador do bloco
data_lenght	U32	4	Tamanho do dado em blocos
time_delta	U32	4	Δt entre escrita e leitura (ms)
number_errors	U32	4	Número de erros detectados
corrected_data	U32	4	Quantidade de dados que foram corrigidos
percent_error	F32	4	Taxa de erro detectado em porcentagem
ecc_telemetry	ECC telemetry	30	Dados de telemetria do ECC
mdca_data	MDCA	34	Dados de tempo e posição do MDCA
Tamanho: 88 bytes			

Tabela 34 – Estrutura de dados sub-bloco ECC_Telemetry.

Campo	Tipo	Tamanho (bytes)	Descrição
single_bit_errors	U32	4	Quantidade de erros de 1 bit
double_bit_errors	U32	4	Quantidade de erros de 2 bits
triplepp_bit_errors	U32	4	Quantidade de erros de 3 ou mais bits
left_region_errors	U32	4	Quantidade de erros na região mais à esquerda na palavra
mid_region_errors	U32	4	Quantidade de erros na região central na palavra
rigth_region_errors	U32	4	Quantidade de erros na região mais à direita na palavra
uncorrectable_errors	U32	4	Quantidade de erros não corrigíveis
ecc	U8	1	ECC aplicado
ecc_status	U8	1	Status (0: OK, 1: WARN, 2: FAIL, 3: ERR_UNKNOW)
Tamanho: 30 bytes			

Fluxo de Telemetria (leitura de resultado)

1. **Comando de leitura:** O master envia **NASCERR_CMD** com **cmd_type** = READ_CMD (0x00), informando **block** (página) e **qtd_block** (quantos itens de telemetria deseja receber nesta consulta).
2. **Preparação no NASCERR:** O firmware valida o comando, tira um snapshot da página solicitada e prepara **TXB** com até **N** itens (**N ≤ qtd_block** e **N ≤ MAX_EXPERIMENTS_PER_PAGE**, tipicamente **23**).
3. **Tamanho:** O master lê **IDX** para saber **exatamente quantos bytes** deve ler de **TXB** (*payload* + CRC16).
4. **Leitura dos dados:** o host lê **TXB** com **exp_len** bytes, valida **CRC16**, remove o CRC e divide o payload em blocos de **88 bytes** (**NASCERR_EXPERIMENT**).
5. **Iteração:** se ainda houver mais itens a coletar, o host repete o passo 1 com novo **block**/**qtd_block** (conforme estratégia/paginador).

Detalhamento por fase

1. **Requisição (READ_CMD)**
   - **Frame:** **bytes_msg** = 29 (27 dados + 2 CRC), **cmd_type**=0x00, **scheduler.block**=X, **scheduler.qtd_block**=Y, demais campos 0.
   - **Semântica:**
     - **block** seleciona a **página** de resultados a expor.

- **qtd_block** limita **quantos itens** de telemetria serão preparados em **TXB**.
- O firmware respeita o que houver disponível (ex.: **num_block** exposto via **EXP_SUM_DATA**) e o limite interno (≤**23**).

2. **Preparação (lado NASCERR)**
   - Valida tamanho/limites/CRC do comando.
   - Faz um snapshot atômico da janela solicitada.
   - Monta **TXB**.payload = **N** × **88 B** (com **N** itens **NASCERR_EXPERIMENT**).
   - Calcula CRC16 do *payload* e anexa [MSB][LSB] ao final.
3. **Determinar o tamanho (IDX)**
   - **IDX** (payload 2 B, little-endian) traz **exp_len =** (**N** × 88) + 2.
   - Total de **IDX:** 4 B (2 dados + 2 CRC).
4. **Ler dados (TXB)**
   - Host lê **TXB** com exatamente **exp_len** bytes.
   - Valida CRC16 de **TXB**.
   - Remove os 2 B finais (CRC) e fatia o *payload* em blocos de **88 B**; para cada bloco, desserializa **NASCERR_EXPERIMENT**.
5. **Iterar (se necessário)**
   - Se ainda houver mais itens na página (ou em páginas seguintes), o master reenvia **READ_CMD** ajustando **block**/**qtd_block** até consumir tudo que precisa.

3.6. Fluxo de Realização do Experimento

O fluxo do experimento inicia-se com o envio do telecomando de operação de escrita no qual é tratado por meio de um fluxo de processamento definido. Há inicialmente a validação do comando em que é contemplado:

- Verificação de possíveis colisões entre blocos;
- Checagem da validade do intervalo de índices dos blocos a serem usados no experimento;
- Análise de consistência das repetições;
- Verificação do tipo de escrita solicitado.

Após a validação e passagem do tempo de agendamento inicia-se a escrita na memória SRAM. Ao ser concluída o experimento é dado como em execução (*OP_EXP_RUNNING*). Inicia-se, então, após determinado tempo o ciclo de leitura da SRAM. Há averiguação e registros de falhas e estas dizem a respeito do bloco lido de acordo com (Tabela *34*), ao fim as informações são sintetizadas e aderidas de novo parâmetros de acordo com a (Tabela *33*). Ocorre a alocação de uma página relativa à repetição

atual do experimento na FLASH (database local) do NASCERR. Assim, já pode-se realizar a telemetria destes dados.

Caso seja enviado um telecomando referente a operação de leitura, em que é requisitado uma página já consolidada no database e quantas pacotes de telemetria devem ser entregues, inicia-se o fluxo de telemetria. O NASCERR válida a solicitação, realiza um snapshot dos dados correspondentes da página e prepara uma resposta, respeitando limites internos. Junto aos dados é informado o tamanho exato da resposta, o que permite o OBC saber quanto deve ler. Ao concluir estes processos uma requisição de transmissão é gerada (*REQ_EXP_TX_RESULT*).

Na ponta do OBC há o processo de leitura, verificação de integridade e interpretação, de modo que com a conclusão dos experimentos este ciclo pode ser reiniciado. Após este processo o NASCERR volta ao modo idle de requisições (*REQ_EXP_NO_REQUEST*) e caso seja finalizado todos os experimentos volta ao modo idle operacional (*OP_EXP_IDLE*).

3.7. Processo de Atualização de Firmware

O NASCERR possui um sistema de Bootloader, responsável por inicializar o dispositivo e colocar em execução o firmware da aplicação. Além disso, o Bootloader viabiliza o processo de atualização remota (OTA), com isso possibilitando que uma nova versão do software possa ser carregada sem a necessidade de intervenção física.

O mecanismo de atualização utiliza os dois registros de Bootloader:

- BOOT_DATA: Responsável pela recepção dos pacotes binários do novo firmware;
- BOOT_STAT: Responsável por fornecer informações de estado e feedback do processo de transferência, incluindo as flags de status, processo e versão atual.

Etapas do processo

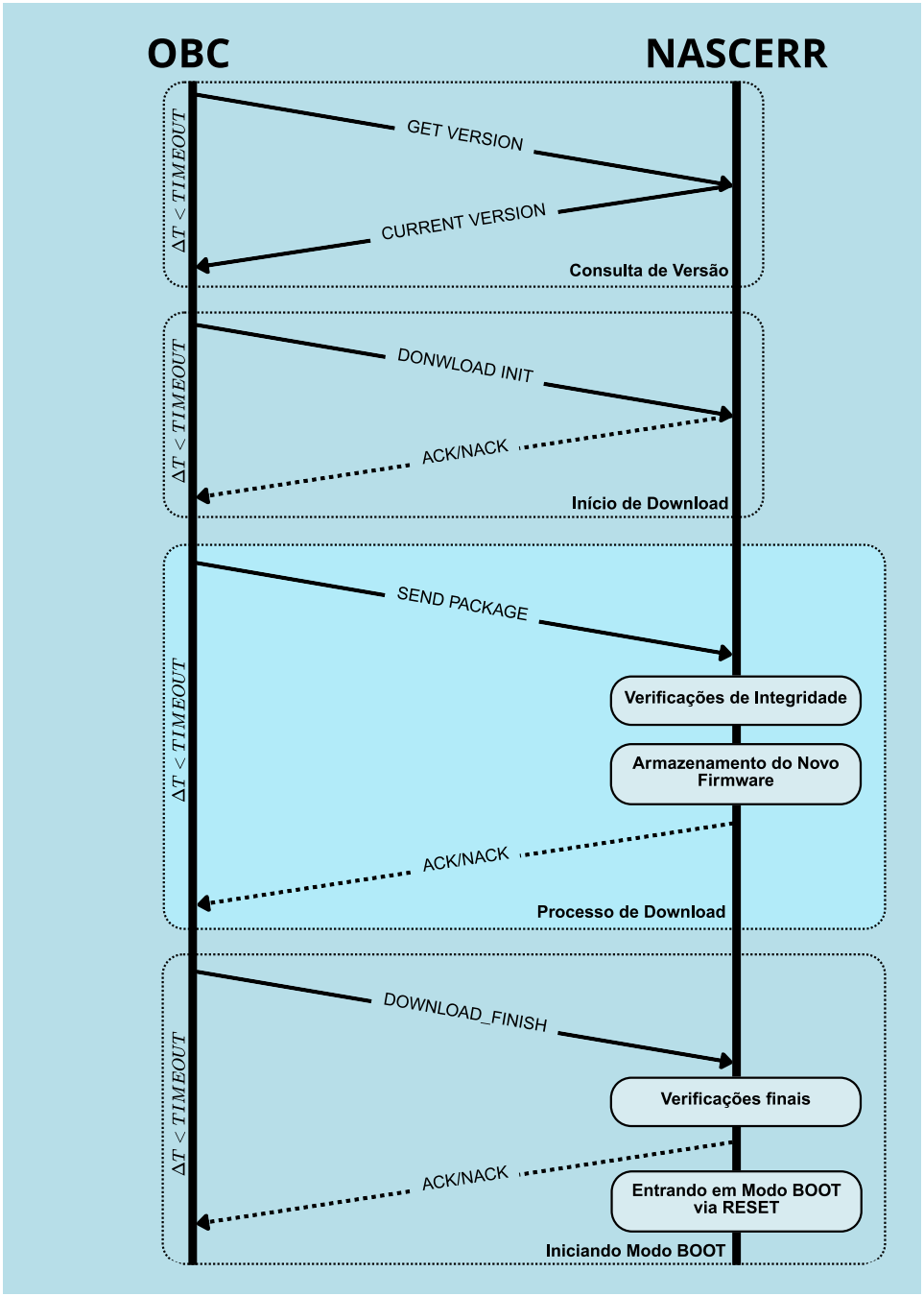
O fluxo completo de atualização é representando na Figura *4*. Esse processo pode ser resumido nas seguintes etapas:

1) **Consulta da versão atual** – Leitura do registro de *BOOT_STAT* para identificação do firmware em execução;
2) **Inicialização do processo de download** – Envio do comando de *DONWLOAD_INIT*, contendo as informações de tamanho total da imagem a ser transferida, versão alvo e número de pacotes que serão transferidos durante esse processo (Tabela *15*).

⚠ **ATENÇÃO**

Após o envio do comando para inicializar o download é necessário realizar o monitoramento do registro de *BOOT_STAT* de forma contínua, mesmo que essa esteja indicando leituras com a flag de *ERROR_DETECTED* ativa ou com a flag *IN_BOOTLOADER* ainda não ativada, pois essas leituras podem estar refletindo os estados anteriores do processo. É recomendado que aguarde nesse estado por até **5 segundos**, caso não ocorra mudanças após esse período considerar como falha.

Figura 4 – Fluxo de Atualização.



3) **Confirmação de entrada em modo de atualização** - Monitorar *BOOT_STAT* até que a flag *IN_BOOTLOADER* seja ativa para dar continuidade ao processo;
4) **Transferência de dados** - Transferência sequencial dos pacotes da imagem via *BOOT_DATA*. De modo que após o envio de cada pacote dever ser consultado *BOOT_STAT* e verificado se as afirmações a seguir são verdadeiras:
    a. Se o último pacote recebido pelo NASCERR corresponde ao último que realmente foi enviado (*last_packet_id*);
    b. Sinalização de sucesso de recepção indicada pela ativação da flag de *DONWLOAD_SUCCESS*;
    c. Ausência de ocorrência de falhas, indicada pela desativação da flag de *ERROR_DETECTED.*
5) **Finalização do processo –** Envio do comando de *DONWLOAD_FINISH*, contendo a versão da imagem que acabou de ser enviada e o hash criptográfico SHA-256 do binário, após esse comando o sistema será reinicializado e o Bootloader ficará responsável por realizar as verificações finais de integridade e colocar a nova versão em execução.

Cálculo do Hash SHA-256

Para realizar a verificação de integridade da imagem recebida, o NASCERR utiliza o periférico HASH do STM32, que segue uma ordem de processamento de dados particular. O cálculo do SHA-256 no sistema de solo replica exatamente esse comportamento, assim, garantido que o código enviado da terra seja idêntico ao calculado pelo microcontrolador nesse processo de verificação.

O algoritmo funciona da seguinte forma:

1) A imagem do firmware é dívida em blocos de 4 bytes;
2) Dentro de cada bloco, a ordem dos bytes é invertida, emulando o tratamento little-endian do periférico;
3) Caso o tamanho da imagem não seja multiplico de 4, o último bloco é preenchido com bytes nulos (0x00) até completar 4 bytes;
4) O hash SHA-256 é calculado sobro a sequência resultante desses blocos invertidos;

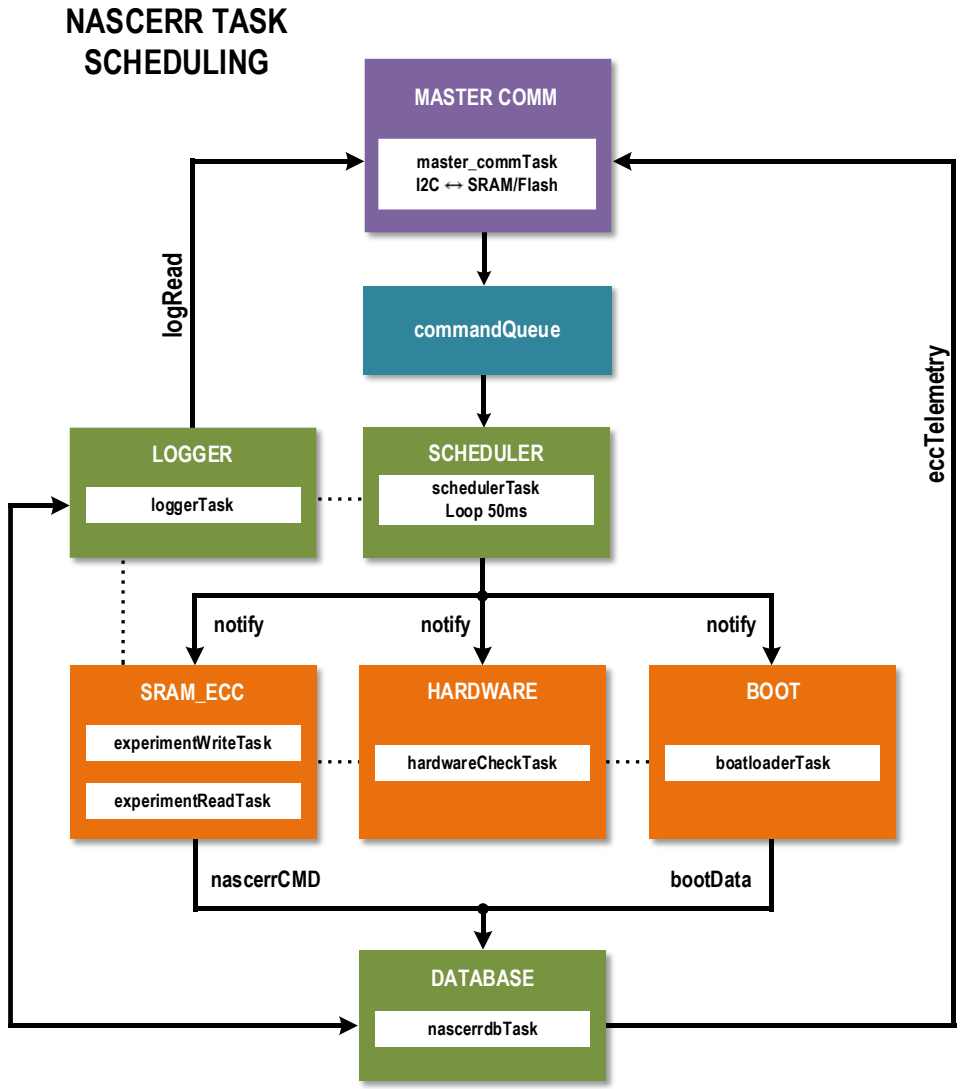
Por exemplo, considere os dados: 01 02 03 04 05 , então:

1. Temos que o primeiro bloco seria os primeiros 4 bytes no formato little-endian: 04 03 02 01

2. O segundo bloco deve então ser completado com zeros e colocado no formato little-endian, ficando dessa maneira: 00 00 00 05
3. Desse modo, a sequência final que deverá ser processada pelo algoritmo SHA-256 será: 04 03 02 01 00 00 00 05

3.8. Interação do Sistema

Figura 5 – Arquitetura do Sistema.



MASTER

O papel do OBC, *MASTER* da comunicação e responsável pelo controle dos experimentos e comunicação com o *SLAVE*, são descritos abaixo:

- **Envio de Telecomandos:** Transmite comandos individuais ou sequenciais para o NASCERR, incluindo operações de escrita/leitura em memória, limpeza de regiões do flash e gravação de dados auxiliares (ex.: GPS).
- **Gerenciamento de Fluxo de Comunicação:** Controla offsets e páginas de leitura, aplica intervalos entre operações e garante a sincronização correta entre envio de comandos e recepção de respostas.
- **Processamento de Telemetria:** Recebe os pacotes de resposta do NASCERR, decodifica-os em formato estruturado e apresenta informações de status, logs e resultados de experimentos.
- **Atualização de Firmware:** Coordena o processo de envio de firmware, fragmentando o arquivo em blocos, validando CRCs e assegurando que a nova versão seja corretamente gravada no NASCERR.

SLAVE

O NASCERR é o *SLAVE*, este é baseado em um microcontrolador *STM32* executando *FreeRTOS*. Ele é responsável pela execução dos experimentos e pelo envio dos dados de telemetria ao *MASTER*. Suas principais funções incluem:

- **Recebimento e Processamento de Comandos:** Interpreta os telecomandos recebidos, valida integridade (CRC) e enfileira-os para execução. Suporta operações de escrita, leitura, limpeza de memória, transição para bootloader e controle de energia (*sleep/wakeup*).
- **Execução de Experimentos:** Realiza gravação e leitura em SRAM, aplica padrões de dados, coleta estatísticas de erros e correções, e registra resultados no banco de dados interno.
- **Gerenciamento de Hardware:** Monitora corrente, temperatura e outros parâmetros internos, sinalizando falhas e atualizando registros periódicos de telemetria.
- **Atualização de Firmware:** Executa a rotina de bootloader quando solicitado pelo MASTER, recebendo pacotes do novo firmware, validando-os e gravando na memória de programa.
- **Escalonamento de Tarefas (FreeRTOS):** Utiliza múltiplas *threads* coordenadas pelo schedulerTask, responsáveis por escrita/leitura de experimentos, checagem de hardware, gravação de logs e operação do bootloader. Mutexes garantem integridade no acesso ao banco de dados e à SRAM.

4. ACRÔNIMOS

Termo	Definição
NASCERR	Nanosatellite Error Resilient Radiation Mission
LESC	Laboratório de Engenharia de Sistemas de Computação
UFC	Universidade Federal do Ceará
ROBC	Robust On-Board Computer
LEO	Low Earth Orbit (Órbita Baixa da Terra)
FPGA	Field Programmable Gate Array
ECC	Error Correction Code (Código de Correção de Erros)
SRAM	Static Random-Access Memory
I2C	Inter-Integrated Circuit
ADDR	Address
STAT	Status
IDX	Index
TXN	Transaction
CMD	Command
MOD	Mode
CRC	Cyclic Redundancy Check
GPS	Global Positioning System
TEMP	Temperature
ERROR_EO	Operation Error
ERROR_EPS	Power Supply Error
ACK	Acknowledge
OBC	On-Board Computer
TBEC	Triple Bit Error Correction
MRSC	Multiple Redundancy Single Correction
FILTER	Image Filter
OTA	Over-The-Air

COMP	Compression
LOSSLESS	Lossless Compression
LOSSY	Lossy Compression
F32	32-bit Floating Point
U32	Unsigned 32-bit Integer
U16	Unsigned 16-bit Integer
U8	Unsigned 8-bit Integer

5. REFERÊNCIAS

[1] A. Edpuganti, V. Khadkikar, M. S. Elmoursi, H. Zeineldin and M. Al Hosani, "A Novel EPS Architecture for 1U/2U Cubesats with Enhanced Fault-Tolerant Capability," 2020 IEEE Industry Applications Society Annual Meeting, Detroit, MI, USA, 2020, pp. 1-6, doi: 10.1109/IAS44978.2020.9334817.

[2] D. Paiva, R. Lima, M. Carvalho, F. Mattiello-Francisco and H. Madeira, "Enhanced software development process for CubeSats to cope with space radiation faults," 2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing (PRDC), Beijing, China, 2022, pp. 78-88, doi: 10.1109/PRDC55274.2022.00022.

[3] H. d. S. Castro, J. A. N. da Silveira, A. A. P. Coelho, F. G. A. e Silva, P. d. S. Magalhães and O. A. de Lima, "A correction code for multiple cells upsets in memory devices for space applications," 2016 14th IEEE International New Circuits and Systems Conference (NEWCAS), Vancouver, BC, Canada, 2016, pp. 1-4, doi: 10.1109/NEWCAS.2016.7604783.

[4] F. Silva, A. C. Pinheiro, J. A. N. Silveira and C. Marcon, "A Triple Burst Error Correction Based on Region Selection Code," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 31, no. 8, pp. 1214-1222, Aug. 2023, doi: 10.1109/TVLSI.2023.3273085.

[5] P. Fortescue, J. Stark, and G. Swinerd, *Spacecraft Systems Engineering,* 3rd edition. John Wiley and Sons, 2003.

[6] R. W. Hamming, "Error detecting and error correcting codes," *The Bell System Technical Journal,* vol. 29, no. 2, pp. 147-160, 1950.